

Make Your LLM Fully Utilize the Context

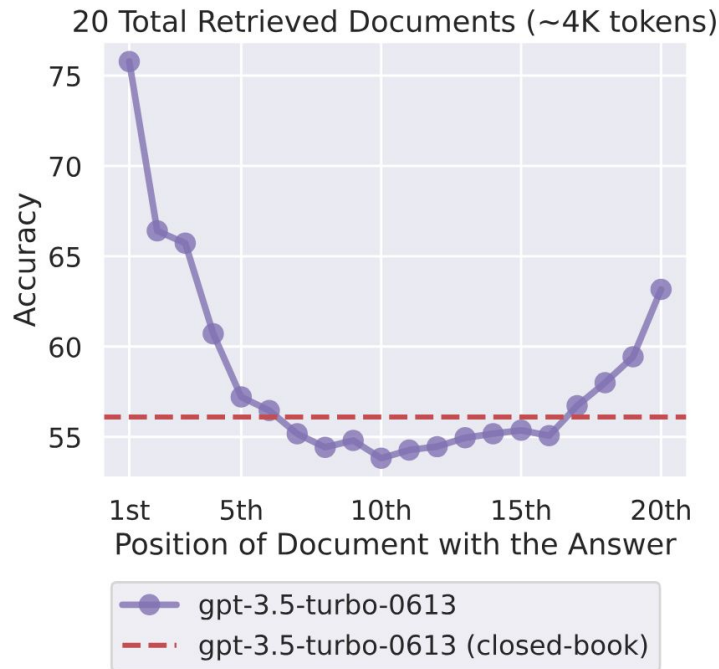
윤세환

목차

- 사전 지식
- 배경
- 방법
 - 훈련: Information-Intensive Training
 - 평가 방법: Long-Context Probing
- 실험 결과 및 분석

사전 지식 - Lost in the middle

- LLM의 발달로, **RAG**가 등장하였으며, **RAG**는 질문과 동시에 **K**개의 문서를 같이 입력해주는 기술을 의미
- 질문의 정답과 관련이 있는 문서가 가운데에 배치될수록 성능이 더 저하된다고 함
- 반대로, 관련있는 문서가 입력의 초반부 혹은 후반부에 배치될 경우에는 성능이 높음

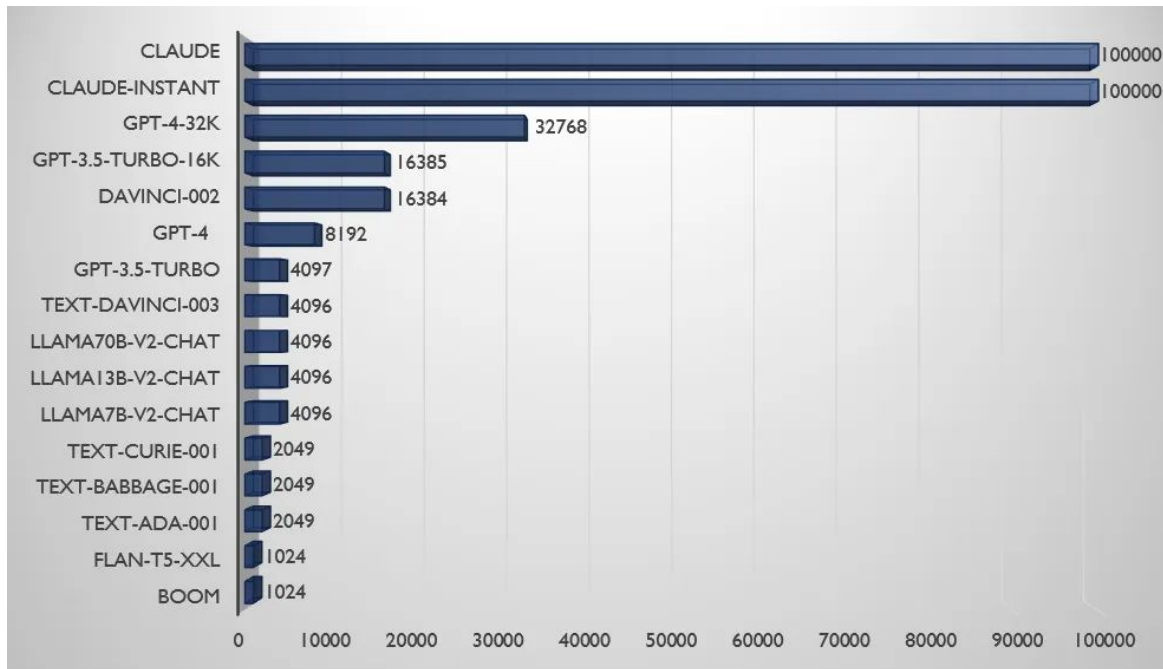


배경

- LLM 모델에서 **training context windows** (학습시 한번에 고려되는 토큰의 개수)가 점점 확대되고 있는 추세
- **context windows**가 더 클수록, 더욱 긴 문장에 대한 답변과 같은 현실의 구체적인 문제를 해결하는데 더욱 도움이 될 수 있음

배경

- 2023년 기준 독점 LLM과 오픈소스 LLM들의 context window size 현황



배경

- 그러나 최근 이런 **long-context** (context window가 큰 모델)들은 **lost-in-the-middle** 문제에 마주쳐있는 상황
- 본 논문에서는 이 문제의 주요 원인을 의도치않은 학습 데이터의 편향이라고 주장
- 이러한 가정을 기반으로 모델에게 중요한 정보는 전체 컨텍스트 내 어디에도 존재할 수 있음을 강조해서 학습하는 **INformation-INtensive training** 방식을 제안

방법 - Information-Intensive Training

- **raw text**를 128 토큰으로 쪼갠 후, 쪼개진 세그먼트중 하나에 대해 질문과 대답을 생성한다. 이후 해당 세그먼트와 다른 랜덤한 **raw text**들을 조합하여 하나의 큰 **long context**를 생성 (fine-grained, 세분화)

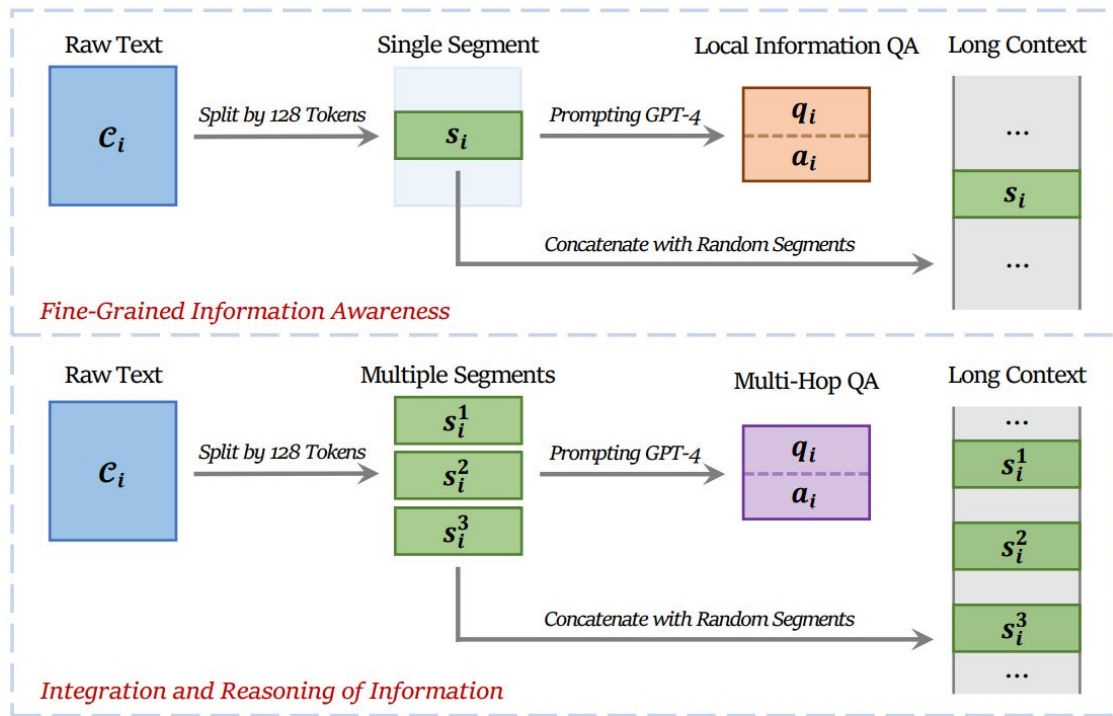
$$(q_i, a_i) \sim \text{Prompting}(s_i, I_f; \text{LLM}), \quad \mathcal{L}_i = \oplus \{\text{Shuffle}(s_i, [r_j])\}$$

- 긴 문장 내 주요 정보가 넓게 흩어진 경우를 학습시키기 위해, **raw text**에서 여러 세그먼트를 가져와 이를 활용하여 질문과 대답을 생성한 후, 다른 **raw text**들과 조합하여 하나의 큰 **long context**를 생성한다. 이 때 가져온 세그먼트들은 **long context** 내 랜덤하게 배치된다.

(Integration and reasoning of information, 정보의 통합과 추론)

$$(q_i, a_i) \sim \text{Prompting}([s_i], I_r; \text{LLM}), \quad \mathcal{L}_i = \oplus \{\text{Shuffle}([s_i], [r_j])\}$$

방법 - Information-Intensive Training



방법 - Information-Intensive Training

- 학습 데이터의 길이 편향을 막기 위해서, **context**의 길이는 **4K~32K** 사이를 균등하게 분포하도록 설정
 - 짧은 맥락에도 좋은 성능을 유지하기 위해서, 세그먼트를 추출한 원본 텍스트도 학습에 사용
- 학습 데이터 구성
- **fine-grained long-context** 데이터 : 1.1M(63%)
 - **integration and reasoning of information long-context** 데이터 : 300K(17%)
 - 원본 텍스트 : 150K(9%)
 - **general instruction-tuning data** : 200K(11%)

위 데이터를 사용하여 **Mistral-7B-Instruct-v0.2**를 파인 튜닝하였고,
이를 **FILM-7B**라 명칭

방법 - Long-Context Probing

- 기존 방식인 **Needle-in-the-Haystack**의 경우, 모델이 긴 **context**에 존재하는 정보를 잘 활용하는지에 대한 평가 방식
 - 이 방식 덕분에, GPT-4나 Claude 2.1같은 모델도 **lost-in-the-middle** 문제를 겪고 있음을 발견
- 본 모델에 **Needle-in-the-Haystack**를 적용했을 때 거의 완벽한 성능을 보임
- 그러나, **Needle-in-the-Haystack**에서 거의 완벽한 점수가 나왔다는 것이 곧 LLM이 긴 **context**의 정보를 잘 활용함을 과대평가해서 보여준다고 주장
 - 평가 데이터는 문서 스타일의 데이터를 사용하고 있으며, 이는 일반적인 문장을 사용하여 **pretrain**된 LLM에 대해서는 좋은 수치를 리턴할 것이며, 평가 데이터의 **forward retrieval** 패턴은 긴 **context**의 자료를 사용하는 어려움 자체를 단순화할 수 있다.

방법 - Long-Context Probing

- forward retrieval 패턴
 - 질문에 대한 정답이 질문 내용의 바로 직후에 나오는 패턴을 의미
 - “샌프란시스코에서 가장 좋은 것은 무엇인가?”라는 질문에 대해,
“샌프란시스코에서 가장 좋은 것은 햇볕이 좋은 날 공원에 앉아 샌드위치를 먹는 것이다”같이
질문의 핵심과 그 답변이 바로 직후에 나오는 패턴
- “긴 컨텍스트 내 정보를 찾는 것”보다 “바로 직후에 나오는 정보를 찾는 것”이
더 쉽게 학습될 수 있기에, 본 방식에서 높은 점수를 가진다는 것은
“긴 컨텍스트 내 정보를 찾는 능력”보다 “바로 직후에 나오는 정답을 찾는 것”에
더 치중될 수 있다.

방법 - Long-Context Probing

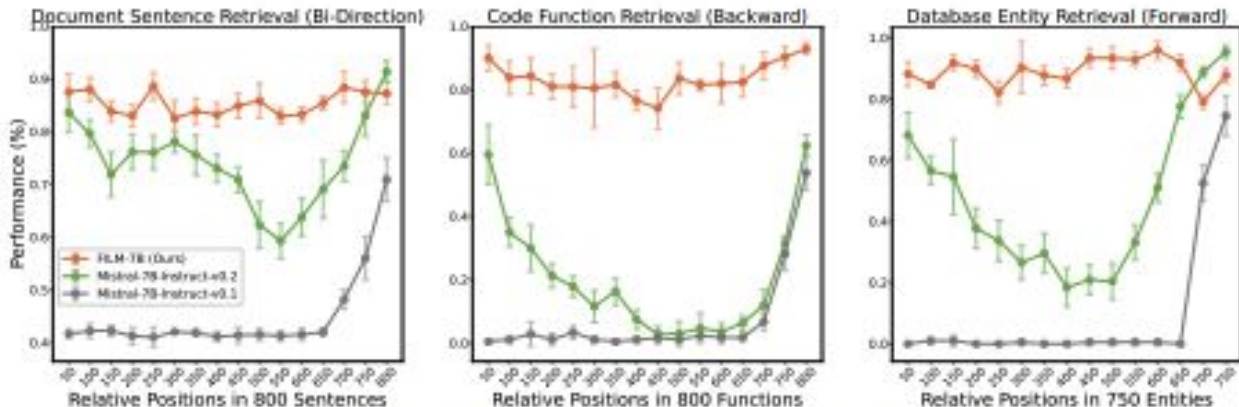
- Needle-in-the-Haystack은 long context LLM 모델의 성능을 적절하게 반영하지 않다고 주장하며, VaL Probing을 제안
- VaL 방식은 3가지의 context style을 사용한다.
(문서 형식, 코드, 데이터베이스와 같은 구조화된 텍스트)
- 이를 통해 3가지의 retrieval 패턴에 대해 평가한다.
(forward, backward, bi-directional)

방법 - Long-Context Probing

- 문서 형식의 데이터 (Bi-Direction)
 - 여러 문장에 걸쳐 질문에 대한 정보를 배치함으로써, 모델이 한 문장뿐만 아니라 앞/뒤에 있는 다른 문장까지 고려해야 하도록 구성
- 코딩 형식의 데이터 (Backward)
 - 특정 기능을 수행하는 코드의 이름을 찾는 테스트를 통해, 역순으로 정보를 가져오는 성능을 평가
- 구조화된 데이터 (데이터베이스 등)
 - ID, Label, description으로 구성된 데이터베이스 엔티티를 사용하며 ID를 주고 그에 대한 label과 description을 찾는 테스트
 - 순서상 ID 이후에 Label, description이 배치되어있기 때문에, 정방향으로 데이터를 찾는 능력을 평가

실험 결과 및 분석

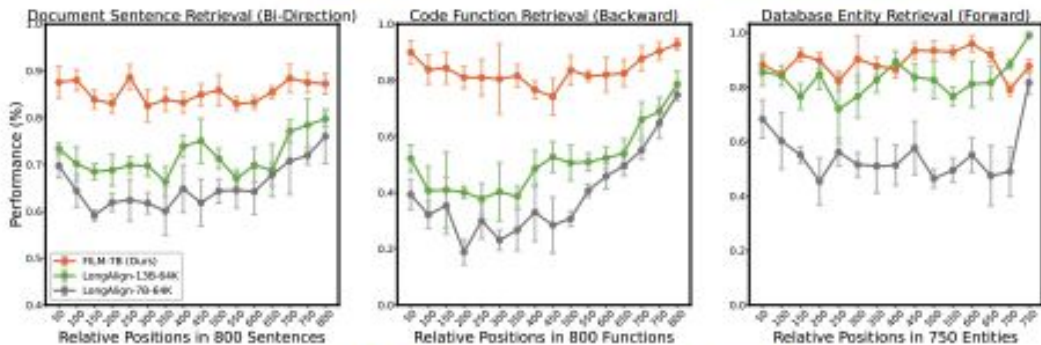
- fine-tuning한 모델 (FILM-7B)와 32K 이상의 context window를 가진 long-context LLM과 비교
- 핵심 문장이 중앙에 있는 케이스에서 기존 모델보다 더 좋은 성능을 보임



(a) Performance of **FILM-7B**, Mistral-7B-Instruct-v0.1, and **Mistral-7B-Instruct-v0.2**.

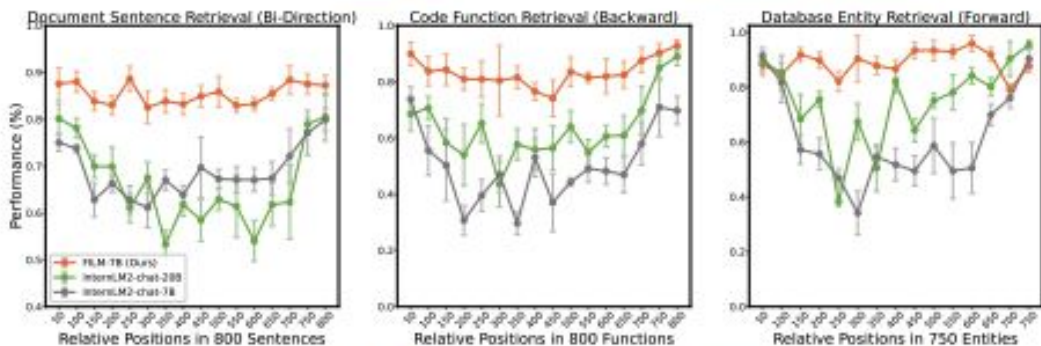
실험 결과 및 분석

- long-context 모델 중 sota인 LongAlign과 InternLM2 모델과 비교한 결과



(b) Performance of **FiLM-7B**, **LongAlign-7B-64K**, and **LongAlign-13B-64K**.

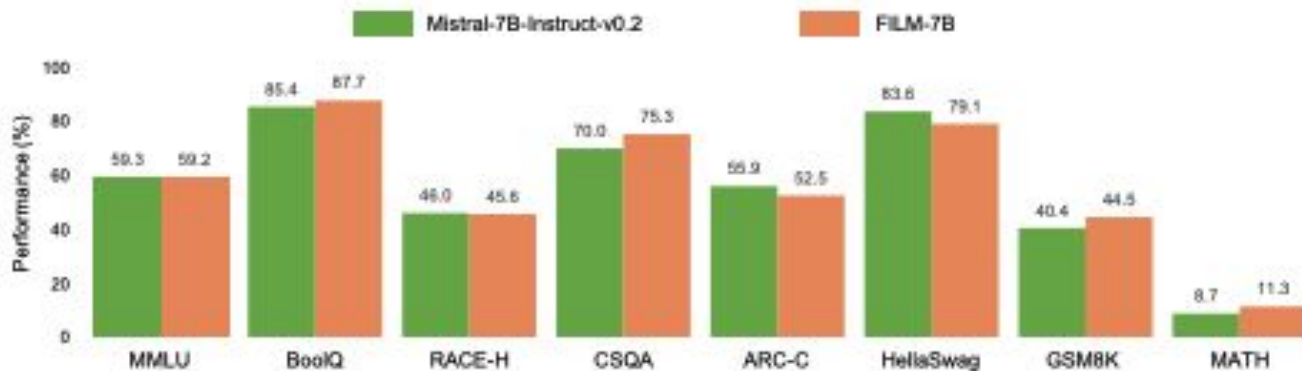
- VaL Probing 방식이 long-context llm 모델에 대해 더 엄격한 평가 지표라고 주장



(c) Performance of **FiLM-7B**, **InternLM2-chat-7B**, and **InternLM2-chat-20B**.

실험 결과 및 분석

- fine-tuning한 모델 (FILM-7B)와 backbone 모델간의 short-context 성능 비교
- short-context task에 대해서도 기존 모델에 준하는 성능을 보임



참고 자료

- Lost in the middle 정리 블로그 : <https://heygeronimo.tistory.com/101>
- Make Your LLM Fully Utilize the Context : <https://arxiv.org/abs/2404.16811>